System Design
Disaster Event Intelligence API

1. Purpose

This system is a backend API for storing, enriching, and analysing disaster event data. It combines locally managed records with data ingested from public external sources.

- SQL-backed API with CRUD support
- Separation of event, source, and ingestion data
- External data ingestion capability
- Analytics over local data

2. High-Level Architecture

Client → FastAPI → Authentication → Service Layer → ORM → Database

External APIs (NASA EONET, USGS) feed data into the ingestion layer

3. Architectural Style

The system follows a layered architecture with clear separation of concerns.

- api/routes – HTTP handling
- api/deps – authentication and dependencies
- crud – data access logic
- models – ORM entities
- schemas – request/response validation
- db – database configuration

4. Core Design Decision

The system generalises disaster events instead of focusing only on wildfires.

- disaster_events
- source_metadata
- ingest_runs

5. Main Components

- FastAPI application entry
- Authentication layer (Basic Auth)
- Events module (CRUD)
- Ingestion module (external APIs)
- Analytics module (insights)

- Health module (monitoring)

6. Database Design

- disaster_events – core event data
- source_metadata – source information
- ingest_runs – ingestion history

7. Data Flow

Manual flow: Client → API → Database

Ingestion flow: Client → API → External API → Transform → Database

Analytics flow: Client → API → Database → Aggregation → Response

8. Security

- HTTP Basic Authentication
- Protected routes: events, ingest, analytics
- Public routes: health endpoints

9. Error Handling

Uses standard HTTP status codes for consistent error reporting.

- 401 Unauthorized
- 404 Not Found
- 422 Validation Error
- 502 Upstream failure
- 503 Database failure

10. Testing Strategy

- CRUD operations
- Authentication validation
- Input validation
- Ingestion behaviour
- Analytics correctness

11. Deployment

- FastAPI with Uvicorn
- SQLite database

- PythonAnywhere deployment

12. Design Trade-offs

- FastAPI vs Django (lightweight)
- SQLite vs PostgreSQL (simplicity)
- Ingestion vs live API (stability)
- Basic Auth vs JWT (simplicity)

13. Scalability

- Move to PostgreSQL
- Add async ingestion jobs
- Introduce caching
- Add geospatial indexing

14. Future Extensions

- Anomaly detection
- Role-based access control
- Dashboard support
- Scheduled ingestion

15. Conclusion

The system uses a layered architecture with ingestion and analytics capabilities, going beyond a basic CRUD API and demonstrating clear architectural reasoning.